AmberCash PH Micro Lending Corp.

Loan Management System (LMS)

Developer Onboarding Workshop Guide

A Module-by-Module Technical Walkthrough

Stack: Next.js • React • Redux • Hasura • GraphQL • PostgreSQL
Deployment: Docker • Nginx • DigitalOcean

Version 1.0 • April 2026

How to Use This Guide

This guide walks new developers through the AmberCash LMS codebase in logical layers — from project setup to business-critical modules. Each module includes a concept overview, key files, hands-on tasks, and review questions.

Workshop Format

Session	Format	Recommended Audience
Self-Paced	Read + explore code independently	Individual developers
Group Workshop	Facilitate with a senior dev presenting each module	New team (2–5 devs)
Sprint Onboarding	Complete 2 modules per day over one week	Joining mid-project

Learning Path Overview

Module	Title	Duration	Focus Area
0	Git Basics & Workflow	2 hrs	Clone, branch, push, pull, dos and don'ts
1	Project Setup & Architecture	2 hrs	Environment, stack overview
2	Authentication & Role System	2 hrs	Auth, Redux, RouteGuard
3	Data Layer: Hasura & GraphQL	3 hrs	GraphQL, graph.provider, native queries
4	Next.js API Routes & Patterns	2 hrs	apiHandler, fetchWrapper
5	Organization Hierarchy	2 hrs	Division → Branch → Client
6	Loan Lifecycle	3 hrs	Application, approval, collection
7	Daily Collection Sheet (DCS)	3 hrs	Core reporting, Excel export
8	Cash Collections & saveV2	2 hrs	Upsert, batch mutations
9	Dashboard & Role-Scoped Views	2 hrs	Status endpoint, widgets
10	General Ledger	2 hrs	Management accounts, GL formulas
11	Fund Transfers & Other Transactions	2 hrs	Multi-step workflows
12	Deployment & Infrastructure	2 hrs	Staging (PM2) vs Production (Docker)

MODULE 0

Git Basics & Workflow

Duration: 2 Hours • Difficulty: Beginner

0.1 What is Git?

Git is a version control system that tracks changes to code over time. Every developer on the LMS team pushes code to the same central repository on GitHub. Git prevents overwriting each other's work and gives a full history of every change.

Key Idea: Think of Git as a "save system" for code. Branches are parallel timelines — work on your own branch without touching the main one until your code is ready.

0.2 First-Time Setup

```
# Set your identity (run once on your machine):
git config --global user.name  "Your Full Name"
git config --global user.email "you@ambercashph.com"

# Verify:
git config --global --list
```

0.3 Cloning the Repository

Cloning downloads the entire repository (code + full history) to your local machine. The LMS uses SSH authentication — add your SSH key to your GitHub account first.

```
# Clone via SSH (recommended):
git clone git@github.com:evalle012006/lms.git

# Enter the project folder:
cd lms

# Switch to the main working branch:
git checkout migration
```

0.4 Core Daily Commands

Command	What It Does	When to Use
git status	Shows which files have changed	Before any commit — run this first
git pull origin migration	Downloads the latest code from GitHub	Start of every working day
git add .	Stages all changed files for commit	After finishing a feature or fix
git add src/file.js	Stages one specific file	When you only want part of your changes

git commit -m "message"	Saves staged changes with a description	After staging files
git push origin your-branch	Uploads your commits to GitHub	When feature is ready for review
git log --oneline	Shows recent commit history	To review recent changes
git diff	Shows exactly which lines changed	Before staging to review your own work

0.5 The Golden Rule: One Branch Per Task

Every feature, bug fix, or improvement must live in its own dedicated branch. Never combine two unrelated tasks in a single branch — it makes code review confusing and rollback impossible.

🔪 **Non-Negotiable Rule:** Before writing a single line of code, create a branch. No exceptions — not even for "tiny" one-line fixes. Every task = one branch = one Pull Request.

Branch Naming Convention

Use the pattern: type/short-description-using-dashes. Keep it lowercase with no spaces.

Type	Use For	Example Branch Name
feature/	New functionality or enhancement	feature/add-csf-column-dcs
fix/	Bug fix for a known issue	fix/cash-collection-duplicate-insert
hotfix/	Urgent fix for a production bug	hotfix/summary-panel-colspan-error
chore/	Refactoring, cleanup, config, docs	chore/add-csf-to-tailwind-safelist

Branch Commands

```
# Always start from an up-to-date migration branch:
git checkout migration
git pull origin migration

# Then create your branch:
git checkout -b feature/your-feature-name

# List all branches (local + remote):
git branch -a

# Switch between branches:
git checkout migration
```

0.6 Typical Daily Workflow

Step	Command	Purpose
1	git checkout migration	Go to the main branch
2	git pull origin migration	Get the latest code

3	git checkout -b feature/my-task	Create your own branch
4	... write code ...	Do your work
5	git status	Check what changed
6	git add .	Stage all changes
7	git commit -m "feat: describe what you did"	Save with a description
8	git push origin feature/my-task	Upload to GitHub
9	Open a Pull Request on GitHub	Request a code review before merging

0.7 Keeping Your Branch Up to Date

While you work on your branch, others keep pushing to migration. Sync regularly to avoid large conflicts.

```
# While on your feature branch:
git fetch origin
git merge origin/migration

# If there are conflicts, Git marks them in the files.
# Resolve the markers, then:
git add .
git commit -m "merge: sync with migration"
```

0.8 Writing Good Commit Messages

A good commit message tells the next developer what changed and why in one line. Use the format: type: short description.

Type	Use For	Example Message
feat:	New feature or enhancement	feat: add CSF withdrawal column to DCS table
fix:	Bug fix	fix: prevent duplicate cash collection on retry
chore:	Refactoring, cleanup, config	chore: add CSF classes to tailwind safelist
docs:	Documentation updates	docs: add API route patterns to README
hotfix:	Urgent production fix	hotfix: correct colSpan in SummaryPanel total row

0.9 Dos and Don'ts

✅ DOs	❌ DON'Ts
Pull before you start working every day	Push directly to the migration branch
Create a new branch for every feature or fix	Work directly on migration
Write a clear, descriptive commit message	Use vague messages like "fix" or "update"
Commit small and often (logical chunks)	Commit everything in one massive commit
Open a Pull Request and request a review	Merge your own PR without a review

Resolve conflicts locally before pushing	Force-push (git push --force) to a shared branch
Check git status before staging	Accidentally stage unrelated files
Delete merged branches to keep things clean	Leave stale branches sitting forever

0.10 Handling Merge Conflicts

A merge conflict happens when two people edited the same lines in the same file. Git cannot decide automatically — you must choose which version to keep.

```
# Git marks conflicts like this:
<<<<<<< HEAD  (your changes)
const x = 'your version';
=======
const x = 'their version';
>>>>>>> origin/migration  (incoming changes)

# To resolve:
# 1. Open the file, decide which version to keep (or combine both)
# 2. Delete all <<<, ===, >>> markers
# 3. git add the resolved file
# 4. git commit
```

0.11 Creating a Pull Request (PR) on GitHub

A Pull Request is how your code gets reviewed before it is merged into migration. You push your branch to GitHub, then open a PR to propose the merge. Every PR must be reviewed and approved before merging — never merge your own PR.

Step-by-Step: Opening a PR

Step	What to Do	Details
1	Push your branch to GitHub	git push origin feature/your-branch-name
2	Open GitHub in your browser	Go to github.com/evalle012006/lms
3	Click Compare & pull request	GitHub shows this banner automatically after a push
4	Set the base branch to migration	This is the branch your code will be merged INTO
5	Write a clear PR title	Same format as commits: feat: add CSF column to DCS
6	Fill in the PR description	Explain what changed, why, and how to test it
7	Assign Reviewer: evalle012006	Click the Reviewers gear icon on the right sidebar
8	Assign yourself under Assignees	Shows who owns and is responsible for the branch
9	Add a Label (optional)	feature, bug, chore, hotfix — helps filter PRs
10	Click Create pull request	PR is now open and the reviewer is notified by email

Writing a Good PR Description

A clear PR description saves the reviewer time and reduces back-and-forth. Always include these four sections:

- What changed — A brief description of the feature or fix

- Why — The reason for the change (bug report, task, business requirement)
- Files changed — List the key files you modified and what you changed in each
- How to test — Step-by-step instructions for the reviewer to verify the change works correctly

Assigning evalle012006 as Reviewer

Every PR must be assigned to the project lead for review. This is required — not optional.

- On the PR page, look at the right sidebar
- Click the gear icon next to Reviewers
- Type evalle012006 in the search box and select the account
- The reviewer receives a GitHub notification and an email automatically
- Wait at least one business day before following up — do not send repeated pings

☐ **Self-Review Checklist Before Requesting Review:** Read every line you changed in the GitHub diff. Confirm: no console.log statements left, no hardcoded credentials, no unrelated files accidentally staged, and the feature works on your local machine.

After the Review

Reviewer Action	What It Means	What You Do
Approved	Code looks good, ready to merge	Merge the PR (Squash and merge), then delete the branch
Changes requested	Reviewer found issues to fix	Fix on the same branch, push again — PR updates automatically
Comment only	Question or suggestion, not blocking	Reply to the comment or fix it, then re-request review

⚠ **Never:** Merge a PR that has not been approved — even if you are in a hurry, even if it is a one-line fix. The review process protects the production system.

0.12 Handling Merge Conflicts

A merge conflict happens when two people edited the same lines in the same file. Git marks the conflict in the file and you must resolve it manually.

```
# Git marks conflicts like this:
<<<<<<< HEAD  (your changes)
const x = 'your version';
=======
const x = 'their version';
>>>>>>> origin/migration  (incoming changes)

# To resolve:
# 1. Open the file, decide which version to keep (or combine both)
# 2. Delete all <<<, ===, >>> markers
# 3. git add the resolved file
# 4. git commit
```

0.13 Review Questions

- What is the difference between git fetch and git pull?
- Why should you never push directly to the migration branch?

- What does git add . do differently from git add src/file.js?
- If a colleague pushed a bug to migration, how would you find which commit caused it?
- What three things must you set on the GitHub PR page before clicking Create pull request?
- What should you do if the reviewer requests changes on your PR?

🏗️ MODULE 1

Project Setup & Architecture

⏱️ Duration: 2 Hours • Difficulty: ⭐ Beginner

1.1 Concept Overview

The LMS is a multi-tenant microfinance web application. The frontend is a Next.js app that talks to two data sources: a set of Next.js API routes (REST-like, server-side) and a Hasura GraphQL engine that auto-generates a GraphQL API on top of PostgreSQL.

🧠 **Key Idea:** Think of the system as three layers: UI (React/Next.js) → API Routes (business logic) → Hasura/PostgreSQL (data).

1.2 Tech Stack at a Glance

Technology	Role in LMS	Where to Look
Next.js 13	Framework: pages, routing, SSR, API routes	next.config.js, src/pages/
React + Redux	UI components and global state	src/components/, src/redux/
Hasura v2.37	Auto-generated GraphQL API on PostgreSQL	database/compose-hasura.yaml
PostgreSQL	Primary database (tables + stored functions)	database/
Tailwind CSS	Utility-first styling	tailwind.config.js
ExcelJS	Client-side Excel file generation	src/components/.../export*.js
Docker + Nginx	Containerized deployment with load balancing	docker-compose.yml, nginx.conf
DigitalOcean Spaces	Object storage for images/files	useSignedUrl hook

1.3 Directory Structure

```
src/pages/          → Next.js pages & API routes (/api/v2/...)
src/components/     → React components, organized by feature
src/lib/            → Utilities: graph provider, fetchWrapper, constants
src/redux/          → Redux store, actions, reducers
database/           → Docker configs, Hasura metadata, migrations
```

1.4 Hands-On Tasks

- Clone the repo: git@github.com:evalle012006/lms.git (branch: migration)
- Copy .env.example to .env and fill in credentials
- Run: npm install && npm run dev
- Open src/pages/_app.js — trace how Layout, Redux store, and RouteGuard wrap every page
- Open next.config.js — note the image domain allowlist and publicRuntimeConfig

1.5 Review Questions

- Where does Hasura get its database connection string?
- What is the difference between NEXT_PUBLIC_* env vars and server-only vars?
- Why are there 10 Docker replicas of the lms service?

MODULE 2

Authentication & Role System

Duration: 2 Hours • Difficulty: ★★ Intermediate

2.1 Concept Overview

Every feature in the LMS behaves differently based on who is logged in. The role system uses a numeric rep field plus a shortCode for fine-grained control. Getting this wrong means showing the wrong data or blocking valid users.

2.2 Role Hierarchy

role.rep	Label	shortCode Examples	Access Scope
1	Super Admin / Admin	(none needed)	Full system access
2	Area/Regional Management	area_admin, regional_manager, deputy_director	Their area/region/division
3	Branch Manager / Cashier	cashier	Their designated branch
4	Loan Officer	(none needed)	Their own groups & clients

2.3 Key Files

- src/components/RouteGuard.js — Redirects unauthenticated users to /login
- src/redux/actions/userActions.js — setUser dispatched on login
- src/redux/store.js — configureStore with next-redux-wrapper for SSR hydration
- src/services/user-service.js — BehaviorSubject that holds current user
- src/pages/api/v2/ — All API routes check req.auth.sub (JWT) or role.rep

2.4 Accessing Role in Components

```
const currentUser = useSelector(state => state.user.data);
const isAdmin       = currentUser?.role?.rep === 1;
const isBranchMgr   = currentUser?.role?.rep === 3;
const isLoanOfficer = currentUser?.role?.rep === 4;
const isCashier     = currentUser?.role?.shortCode === 'cashier';
```

2.5 API-Side Auth Pattern

```
// In any API route:
const userId = req.auth?.sub || req.body.currentUserId;
const user   = await findUserById(userId);
if (user.role.rep > 3) return res.status(403).send({ success: false });
```

⚠ **Known Vulnerability:** req.body.currentUserId is a fallback that bypasses JWT. This is being phased out — always prefer req.auth.sub in new code.

2.6 Hands-On Tasks

- Log in as each role type and compare the navigation menu (src/components/Nav.js)
- Find where MenuItems are filtered by role in Nav.js
- Add a console.log in RouteGuard.js to trace the auth check flow
- Read src/pages/settings/roles/index.js — role management is admin-only (role.rep !== 1 redirects)

2.7 Review Questions

- How does the frontend know if the user is logged in after a page refresh?
- What is the difference between role.rep === 2 with shortCode regional_manager vs area_admin?
- Why does fetchWrapper auto-logout on 403 responses?

🔗 MODULE 3

Data Layer: Hasura & GraphQL

⏱ Duration: 3 Hours • Difficulty: ⭐⭐⭐ Intermediate-Advanced

3.1 Concept Overview

Hasura sits between the app and PostgreSQL. It automatically generates a GraphQL API for every table. The LMS wraps Hasura with a custom abstraction (GraphProvider) so developers never write raw GraphQL strings in components.

3.2 GraphProvider Abstraction

```
// src/lib/graph/graph.provider.ts
const graph = new GraphProvider();


// Single query:
const { data } = await graph.query(queryQl(loansType, { where: { status: { _eq: 'active' } }
}));


// Batch mutation:
const mutationList = [];
mutationList.push(updateQl(loansType, { set: { status: 'closed' }, where: { _id: { _eq: id }
} }));
await graph.mutation(...mutationList);
```

3.3 Key Utilities

Function	File	Purpose
createGraphType()	graph.util.ts	Defines a table + its fields as a typed object
queryQl()	graph.util.ts	Builds a SELECT query with where/limit/order_by
insertQl()	graph.util.ts	Builds an INSERT mutation
updateQl()	graph.util.ts	Builds an UPDATE mutation
aggregateQl()	graph.util.ts	Builds aggregate queries (count, sum, etc.)
graph.apollo.query()	graph.provider.ts	Raw ApolloClient query for complex cases

3.4 Field Definitions

```
// src/lib/graph.fields.js — ALL field definitions live here
// NEVER hardcode field strings in API routes.
// Always check graph.fields.js first before creating new constants.


import { LOAN_FIELDS, CASH_COLLECTIONS_FIELDS } from '@/lib/graph.fields';
const loansType = createGraphType('loans', LOAN_FIELDS)('loans');
```

📌 **Rule:** Always check graph.fields.js for an existing field definition before creating a new one. Duplicate field strings cause silent bugs.

3.5 When to Use Raw Apollo

Use graph.apollo.query() with gql`` directly when you need: _like filters, custom aggregates inside jsonb_agg, or Hasura native queries (stored function calls).

```
// _like is NOT supported by queryQl — use raw apollo:
await graph.apollo.query({ query: gql`query { users(where: { email: { _like: "%@domain.com" } }) { _id } }` });
```

3.6 Hasura Native Queries

Native Queries are a Hasura feature that lets you expose a raw PostgreSQL function or SQL query directly as a GraphQL field. The LMS uses this extensively for complex reporting that cannot be expressed in Hasura auto-generated query builder.

Why Native Queries?

Standard Hasura queries work well for simple CRUD. But reports like the DCS require multi-table joins, window functions, aggregations, and conditional logic that is far more natural in SQL. Native Queries bridge that gap.

How It Works

- 1. Write a PostgreSQL function (or view) that returns a typed rowset
- 2. Register it in Hasura as a Native Query via the Hasura console or metadata YAML
- 3. Hasura generates a GraphQL field for it — queryable like any table
- 4. Call it from the app using graph.apollo.query() with a gql template

```
// Calling a native query (PostgreSQL function via Hasura):
import { gql } from "@apollo/client";

const { data } = await graph.apollo.query({
  query: gql`
    query GetDCS($branch_id: uuid!, $selected_date: date!) {
      fn_get_daily_collection_sheet_v2(
        args: { branch_id: $branch_id, selected_date: $selected_date }
      ) {
        lo_id lo_name group_name mcbu_target mcbu_actual
        csf_collection net_collection
      }
    }
  `,
  variables: { branch_id: branchId, selected_date: date },
  context: { headers: { Authorization: `Bearer ${token}` } }
});
```

Registering a Native Query in Hasura Metadata

Native queries are defined in the Hasura metadata directory. When you create a new PostgreSQL function that will be called as a GraphQL field, it must be registered here.

```
# In database/hasura/metadata/databases/default/
# Add an entry in native_queries/ or logical_models/
```

```
# After editing metadata YAML, apply it:
hasura metadata apply
# Or reload via Hasura console → Metadata → Reload
```

📌 **Key Rule:** If you add or modify a PostgreSQL function that is used as a Native Query, you must also update the Hasura metadata and reload. Otherwise the old schema is served and new columns are invisible to GraphQL.

Key Native Queries in LMS

PostgreSQL Function	GraphQL Field Name	Purpose
fn_get_daily_collection_sheet_v2	fn_get_daily_collection_sheet_v2	Full DCS report for one branch + date
fn_get_cash_collection_with_filter_v2	fn_get_cash_collection_with_filter_v2	Filtered cash collection list
get_lo_transaction_summary	get_lo_transaction_summary	LO-level transaction totals
get_dashboard_summary_view	get_dashboard_summary_view	Dashboard KPI summary

Adding a New Native Query — Step by Step

- Step 1: Write and test your PostgreSQL function in psql or a DB client
- Step 2: Create or update the migration SQL file in database/
- Step 3: Register the function in Hasura — console → Data → Native Queries → Add
- Step 4: Define the Logical Model (return type) so Hasura knows the shape of the result
- Step 5: Export metadata: hasura metadata export
- Step 6: Commit the updated metadata YAML to the repo
- Step 7: Call from the app using graph.apollo.query() with your gql template

3.7 Common Pitfalls

- UNION vs UNION ALL: Always use UNION ALL — UNION silently deduplicates rows.
- Nested aggregates inside jsonb_agg() are invalid PostgreSQL — promote to flat CTEs.
- queryQl condition must be wrapped in { where: ... }, not passed directly.
- Hasura aggregate queries return a list — use .length, not a dedicated .aggregate field.
- Native Query not showing in GraphQL? Check: function exists in DB, metadata is applied, and logical model matches the function return type.

3.8 Hands-On Tasks

- Open Hasura console (local dev) → explore the loans and cashCollections tables
- Find fn_get_daily_collection_sheet_v2 in the Hasura console under Native Queries
- Trace a query from a React component → fetchWrapper → API route → graph.apollo.query()
- Write a queryQl call that fetches active loans for a specific branch
- Add a new field to LOAN_FIELDS in graph.fields.js and verify it returns in the query

MODULE 4

Next.js API Routes & Patterns

Duration: 2 Hours • Difficulty: ★★ Intermediate

4.1 Concept Overview

All server-side business logic lives in src/pages/api/v2/. Each file exports a handler created by apiHandler(). The frontend calls these via fetchWrapper — never directly via fetch() or axios.

4.2 API Handler Pattern

```
// Every API route follows this shape:
import { apiHandler } from '@/services/api-handler';

export default apiHandler({
  get:  listData,   // → GET  /api/v2/this-route
  post: saveData,   // → POST /api/v2/this-route
});

async function listData(req, res) {
  // req.auth.sub = logged-in user ID (from JWT)
  // Always validate role before processing
  res.send({ success: true, data: [...] });
}
```

4.3 fetchWrapper (Frontend)

```
import { fetchWrapper } from '@/lib/fetch-wrapper';
import { getApiBaseUrl } from '@/lib/constants';

// GET:
const res = await fetchWrapper.get(getApiBaseUrl() + 'loans/list?status=active');

// POST (mutation/save):
const res = await fetchWrapper.post(getApiBaseUrl() + 'loans/save', payload);
```

⚠ **Important:** fetchWrapper auto-logs out the user on HTTP 403. For soft failures (validation errors), return status 200 with { success: false, message: "..." } instead.

4.4 Response Conventions

Scenario	HTTP Status	Response Shape
Success	200	{ success: true, data: [...] }
Validation error	200	{ success: false, message: "..." }
Unauthorized (wrong role)	403	{ success: false, message: "..." }

Not authenticated	401	{ success: false, message: "..." }
Server error	500	{ success: false, error: e.message }

4.5 Hands-On Tasks

- Open src/pages/api/v2/settings/holidays/list.js — the simplest possible API route
- Open src/pages/api/v2/transactions/fund-transfer/save.js — a complex route with role checks, validation, and mutations
- Trace a POST from a React component through fetchWrapper to the API route to graph.mutation()
- Add a new GET endpoint that returns a count of active loans for the current branch

MODULE 5

Organization Hierarchy

Duration: 2 Hours • Difficulty: ⭐⭐ Intermediate

5.1 Concept Overview

The LMS models a real microfinance org structure. Almost every query is scoped by where in this tree the current user sits. Getting the hierarchy wrong means data leaks across branches.

5.2 The Hierarchy

Level	Entity	Key Fields	Who Manages It
1 (Top)	Division	divisionId	Admin (rep=1)
2	Region	regionId	Admin / Deputy Director
3	Area	areaId	Area Admin
4	Branch	designatedBranchId	Branch Manager (rep=3)
5	Loan Officer	userId / loId	Branch Manager
6	Group	groupId	Loan Officer (rep=4)
7 (Bottom)	Client	clientId	Loan Officer

5.3 Navigation Pattern in Cash Collections

When an Admin drills down from Division → Region → Area → Branch → LO → Group → Client, the URL query params track position: parentId, grandParentId, viewMode. This pattern repeats across several modules.

```
// router.query structure at Branch level:
{ viewMode: 'division', parentId: 'regionId', grandParentId: 'areaId' }
```

5.4 Scoping Queries by Role

```
// Always scope queries by the user's jurisdiction:
if (user.role.rep === 1) { /* no filter — see all */ }
if (user.role.rep === 2) { where.areaId = { _eq: user.areaId }; }
if (user.role.rep === 3) { where.branchId = { _eq: user.designatedBranchId }; }
if (user.role.rep === 4) { where.loId = { _eq: user._id }; }
```

5.5 Hands-On Tasks

- Open src/pages/transactions/cash-collection/index.js — trace getCurrentLevel()
- Map out what URL query params change as you drill from Division to Group
- Find where designatedBranchId is used to scope a Hasura query in any API route

💰 MODULE 6

Loan Lifecycle

⏱ Duration: 3 Hours • Difficulty: ⭐⭐⭐ Advanced

6.1 Concept Overview

The core business of the LMS. A loan moves through a lifecycle from prospect client creation all the way to full payment or bad debt write-off.

6.2 Loan Status Flow

A loan moves through a fixed lifecycle. Understanding these statuses is essential because many screens, validations, and remarks are gated on them.

Status (loans table)	Description	Who Sets It
pending	Application created by the Loan Officer	Loan Officer (rep=4)
ldf_approved	Approved at branch level (LDF signed)	Branch Manager (rep=3)
active	Loan released and collecting — this is the "approved" state in production	Branch Manager / Admin
closed	Fully paid or offset — loan is done	System on full payment / offset transaction

💡 **Important Naming Note:** What the business calls "Approved" maps to status: active in the database. There is no separate approved status in the loans table — once a loan is approved for release, it is immediately set to active with the loan start date.

Reloan / Reloaner Flow (Two-Table Split)

When a client completes their current loan and takes a new one on the same day (a reloan), the system records this across two tables differently — and this is one of the most important things to understand:

Table	Field Set	Value	Meaning
cashCollections	status	tomorrow	The new loan is not active yet today — it starts tomorrow
cashCollections	remarks.value	reloaner or reloaner-cont	Flags the record as a reloan transaction
loans	status	active	The new loan record is already active in the loans table
loans	startDate	tomorrow's date	The loan starts tomorrow, so today's collection is still under the old loan

⚠️ **Common Confusion:** Developers sometimes expect cashCollections.status and loans.status to always match. They do NOT for reloaners. The cashCollections row says "tomorrow" but the loans

row says "active" with a future startDate. Never derive loan status from the cashCollections table alone.

Full Lifecycle Summary

Stage	loans.status	cashCollections.status	Trigger
Application submitted	pending	(none yet)	LO creates loan application
Branch approves (LDF)	ldf_approved	(none yet)	Branch Manager signs off
Loan released	active	active	Loan is released; collection begins
Daily collection	active	active or completed	LO records daily payment
Reloan transaction day	active (new loan, future startDate)	tomorrow (old loan record)	Reloaner remarks set on cashCollections
Reloan next day	active	active	New loan starts; collection resumes normally
Full payment / Offset	closed	completed	Offset remarks trigger loan closure

6.3 Key Tables

- loans — Master loan record (loanType, amount, status, terms, etc.)
- cashCollections — Daily payment records per client per date
- losTotals — Running totals per loan officer
- transferClients — Client transfer requests between groups

6.4 Loan Application Flow

- LO creates a client (prospect status)
- LO submits a loan application (status: pending)
- Branch Manager reviews → sets status: pending_approved
- Admin/Management approves → status: approved
- Loan released → status: active, cashCollections records begin

6.5 Important Business Rules

- A client can only have one active loan at a time
- Cash collections are tied to a specific date (dateAdded) — upsert by loanId + dateAdded
- MCBU (Mandatory Cash Build-Up) and CSF (Client Savings Fund) are collected alongside loan payments
- bmRevertCount limits how many times a Branch Manager can revert a collection

6.6 Hands-On Tasks

- Open src/pages/transactions/loan-applications/admin-view.js and trace the approval flow
- Find the upsert logic in the cash collection save API (query existing by loanId + dateAdded before INSERT/UPDATE)
- Identify all places where loan status is changed and verify the role check before each transition

MODULE 7

Daily Collection Sheet (DCS)

Duration: 3 Hours • Difficulty: ★★★ Advanced

7.1 Concept Overview

The Daily Collection Sheet is the most-used report in the system. It shows each Loan Officer's group collections for a selected date, including MCBU, CSF, loan payments, fees, and net totals. It has two views (DCS Table and Morning/Afternoon) and can be exported to Excel.

7.2 Key Components

Component	File	Role
DCS Page	src/pages/transactions/daily-collection-sheet/index.js	Parent — fetches data, manages state
DCSTable	src/components/.../DCSTable.js	Main tabular display, all columns
SummaryPanel	src/components/.../SummaryPanel.js	Cashbook summary (receipts/payments)
MorningAfternoonPanel	src/components/.../MorningAfternoonPanel.js	LO remittance vs net collection
exportDCSExcel.js	src/components/.../exportDCSExcel.js	ExcelJS export — DCS + Summary sheets

7.3 Data Flow

- 1. Component mounts → reads currentUser role and branch from Redux
- 2. Calls GET /api/v2/data/get_daily_collection_sheet_v2?branch_id=...&selected_date=...
- 3. API calls fn_get_daily_collection_sheet_v2 PostgreSQL function via Hasura native query
- 4. Response is grouped by Loan Officer → Groups → flattened into rows
- 5. Summary fetched separately via /api/v2/data/get_dcs_summary

7.4 Column Groups

Group	Columns	Notes
MCBU	Target, Actual	Mandatory Cash Build-Up
CSF	Collection	Client Savings Fund — added 2025
Regular Loan	Target, Adv. Pay, Actual	60-day term loans
Other Loan	Target, Adv. Pay, Actual	Weekly term loans
Fees	Admission, LRF, CBHB, Hospi, Other Income	Various fee types

Total Collection	Sum of all receipts	Before deductions
Less Returns	MCBU WD/Return, CSF WD/Return	Deductions from total
Net Collection	Total - Returns	What the LO remits
Info	Renewal, Offset, No. of Clients	Informational only

7.5 Excel Export

exportDCSExcel.js uses ExcelJS to generate a multi-sheet workbook client-side. It builds the DCS sheet with merged cells and color-coded header rows, and a Summary sheet with the cashbook (beginning balance, receipts, payments, closing balance).

📌 **Rule:** When adding new columns to DCS, update: (1) PostgreSQL function, (2) DCSTable.js, (3) SummaryPanel.js, (4) exportDCSExcel.js. Missing any one of these causes misalignment.

7.6 Hands-On Tasks

- Render the DCS page for a branch with data — identify each column group visually
- Open exportDCSExcel.js and find where CSF columns are defined
- Trace the SummaryPanel colSpan fix — understand why the total row needed adjustment
- Add a console.log to fn_get_daily_collection_sheet_v2 API handler to see the raw SQL function response

MODULE 8

Cash Collections & saveV2

Duration: 2 Hours • Difficulty: ★★★ Advanced

8.1 Concept Overview

Cash collection is how daily loan payments are recorded. The CashCollectionDetailsPage is the most-used screen in the entire system — every Loan Officer opens it every working day for every group they handle. The saveV2 endpoint is the most frequently called mutation in the system.

8.2 CashCollectionDetailsPage — Screen Breakdown

The page (src/pages/transactions/daily-cash-collection/client/[uuid].js and weekly equivalent) displays every client in a group as a row. Each row is one cash collection record for today's date.

Column / Field	What It Shows	Notes
Slot No.	Client's seat order in the group	Determines sort order of rows
Client Name	Full name of the borrower	
Amount Release	Loan amount originally released	Editable by BM within allowed window
Target Collection	Expected daily payment (amortization)	Derived from loan terms and amount
Actual Collection	Amount the client paid today	LO enters this field
MCBU Col.	Mandatory Cash Build-Up collected today	Minimum enforced by transactionSettings
CSF Col.	Client Savings Fund collected today	Separate savings track
MCBU Balance	Running total of client's MCBU savings	Cumulative across all collection dates
Loan Balance	Remaining principal owed	Decreases with each payment
Past Due	Amount overdue from missed payments	Computed field
Mispayment	Yes/No flag for incomplete payment	Triggers past due tracking
No. of Mispay	Count of missed payment days	
Remarks	Transaction type / special status	See section 8.3 below
Total Collection	Sum of all collectibles for the row	

8.3 Remarks — The Most Critical Field

Remarks tell the system what kind of transaction is happening. The wrong remark causes validation errors, incorrect loan balance calculations, and broken status transitions. Every Loan Officer must understand these.

Normal Payment Remarks

Remark Label	Internal Value	When to Use
Double Payment	double payment	Client paid exactly 2x the target in one day
Advance Payment	advance payment	Client paid more than 2x target (creates advance days)
Past Due	past due	Client missed a previous payment; paying partial today
Past Due Collection	past due collection	Collecting specifically for a past due amount
Matured Past Due	matured-past due	Loan has reached term end but still has outstanding balance
Matured Past Due Collection	matured_past_due_collection	Collection on a matured past due account

Excused Remarks (No Penalty)

Remark Label	Internal Value	When to Use
Good Excused due to Advance Payment	excused advance payment	Client has advance days banked; excused today
Excused Due to Calamity	excused-calamity	Natural disaster or community event
Excused - Hospitalization	excused-hospital	Client is hospitalized
Excused - Death of Client/Family	excused-death	Bereavement excused absence

Mispayment / Delinquent Remarks

Remark Label	Internal Value	Effect
Delinquent	delinquent	Marks client as delinquent; zeros target collection; adds to loan balance
Delinquent Client for Loan Collection	delinquent-offset	Collecting from delinquent account toward offset
Delinquent Client for MCBU	delinquent-mcbu	Collecting only MCBU from delinquent account
Overstated Collection	collection-overstated	Correction: previous collection was too high
Understated Collection	collection-understated	Correction: previous collection was too low

Reloan Remarks (Full Payment + New Loan Same Day)

These remarks are only available when the loan balance reaches zero. They signal the end of the current loan and the beginning of a new one.

Remark Label	Internal Value	Loan Type	What Happens
Reloaner	reloaner	Weekly loan	Final payment made; new loan starts tomorrow; cashCollections.status = tomorrow

Reloaner Cont/MCBU	reloaner-cont	Daily loan	Reloan with MCBU continuity; same tomorrow status split
Reloaner WD/MCBU	reloaner-wd	Daily loan	Reloan with MCBU withdrawal instead of carry-over

💡 **Reloan Rule:** When a Reloaner remark is saved, cashCollections.status is set to tomorrow for that row, but a NEW loans record is inserted with status: active and startDate = tomorrow. The LO will see the new loan appear the following day.

Offset Remarks (Loan Closure)

Offset means the loan is being closed. These remarks trigger loans.status to be set to closed.

Remark Label	Internal Value	When to Use
For Close/Offset - Good Client	offset-good	Client completed loan in good standing; MCBU returned
For Close/Offset - Delinquent Client	offset-delinquent	Closing a delinquent account; MCBU used to settle balance
For Close/Offset - Unclaimed Amount	offset-unclaimed	Client has unclaimed MCBU; loan is closed with unclaimed tracking
For Close/Offset - Matured PD Client	offset-matured-pd	Closing a matured past due account

Validation Rules Enforced by the System

- Actual collection must be divisible by the target amount — unless the remark is past due, excused, or delinquent
- Double Payment remark required when collection = exactly 2x target
- Advance Payment remark required when collection > 2x target
- Reloaner or Offset remark required when loanBalance reaches zero
- Offset remark requires full balance cleared first — cannot offset with remaining balance
- Matured PD accounts must use offset-matured-pd remark, not a regular offset remark
- Past due amount must be >= target collection when past due remark is used

8.4 Upsert Logic (Critical)

A cash collection record is uniquely identified by loanId + dateAdded. On save, the API must:

- 1. Query existing record by loanId + dateAdded
- 2. If found → UPDATE the existing record
- 3. If not found → INSERT a new record
- 4. Never use a plain INSERT without checking first — causes duplicates

```
// Simplified upsert pattern:
const existing = await findCashCollections({ loanId: { _eq: loanId }, dateAdded: { _eq: date } });
if (existing.length > 0) {
  await graph.mutation(updateQl(ccType, { set: payload, where: { _id: { _eq: existing[0]._id } } }));
} else {
  await graph.mutation(insertQl(ccType, { objects: [{ ...payload, _id: generateUUID() }] }));
}
```

8.5 JSON Double-Parse Bug (Fixed)

On retries, the request body was being double-JSON-parsed because middleware parsed it once and the handler parsed it again. Always trust that req.body is already a JavaScript object in apiHandler — do not call JSON.parse(req.body).

8.6 bmRevertCount

Branch Managers can revert a collection (change a submitted collection back to editable). The bmRevertCount field on the cash collection record limits how many times this is allowed. Check this field before allowing a revert.

8.7 Batch Mutations Pattern

```
const mutationList = [];
// Push each mutation with a unique alias:
mutationList.push(updateQl(loansType, { set: {...}, where: {...} }));
mutationList.push(updateQl(groupsType, { set: {...}, where: {...} }));
// Execute all in a single Hasura request:
await graph.mutation(...mutationList);
```

⚡ **Performance:** Always batch related mutations into a single graph.mutation() call. Multiple sequential mutations cause N network round-trips and can leave data in an inconsistent state if one fails.

8.8 Hands-On Tasks

- Open src/lib/constants.js and read through LOR_DAILY_REMARKS and LOR_WEEKLY_REMARKS — notice which remark values are strings vs prefixes
- In daily-cash-collection/client/[uuid].js, find the remarks change handler and trace what happens when reloaner-cont is selected
- Find where cashCollections.status is set to tomorrow and compare it to where loans.status is set
- Find the upsert logic in src/pages/api/v2/transactions/cash-collection/ save endpoint
- Simulate a retry by calling the save endpoint twice with the same loanId + dateAdded — verify only one record exists
- Find where bmRevertCount is checked and incremented

MODULE 9

Dashboard & Role-Scoped Views

Duration: 2 Hours • Difficulty: ⭐⭐ Intermediate

9.1 Concept Overview

The dashboard is the first screen users see after login. Its content is entirely determined by role. Admins see company-wide summaries; branch managers see their branch; loan officers see their collections.

9.2 Status Endpoint

The /api/v2/dashboard/status endpoint is role-scoped — it filters PostgreSQL function get_dashboard_summary_view by the requesting user's jurisdiction (branchId for rep=3, areaId for rep=2, etc.).

9.3 Sub-Components

Component	Purpose
ColorDot	Visual status indicator (green/red/yellow)
CollapsiblePanel	Expandable section for KPI groups
CustomSelect	Reusable dropdown used across filters
CardItem	Individual metric card
CompanyActivitiesSlider	Image carousel for company announcements (DigitalOcean Spaces)

9.4 Date Rollback Logic

The dashboard defaults to the most recent business day. If today is a weekend or holiday, it rolls back to the last working day. This is done with lazy useState initialization to avoid SSR hydration mismatches.

```
// Lazy initialization — runs only on mount, not on every render:
const [currentDate, setCurrentDate] = useState(() => getLastBusinessDay());

// currentDate also in useEffect dependencies:
useEffect(() => { fetchDashboardData(currentDate); }, [currentDate]);
```

9.5 DigitalOcean Spaces & Signed URLs

Company activity images are stored in DigitalOcean Spaces (private bucket). Access uses signed URLs generated server-side. The useSignedUrl hook reads the JWT from Redux state.

9.6 Hands-On Tasks

- Log in as each role and compare dashboard content
- Open /api/v2/dashboard/status and trace how the where clause changes per role

- Upload a test image via CompanyActivitiesSlider and verify it appears in DigitalOcean Spaces

📈 MODULE 10

General Ledger (GL)

⏱ Duration: 2 Hours • Difficulty: ★★★ Advanced

10.1 Concept Overview

The General Ledger (GL) module generates management-level financial statements. Accounts have debit/credit columns with formula-driven computed totals. The system supports account references (one account's value used in another's formula).

10.2 Formula System

Each account can have a formula for its debit, credit, and total_balance columns. Formulas support:

- Own-row variables: debit, credit, prev_balance, total_balance
- Account reference variables: {slug}_{suffix} — e.g., loan_receivables_debit
- Operators: + - * / and parentheses

```
// Example formula:
total_balance = prev_balance + debit - credit

// Cross-account reference:
total_balance = loan_receivables_total_balance + advances_total_balance
```

10.3 Formula Validator

src/pages/api/v2/management-transactions/account-names/save.js contains the validateFormula() function. It sanitizes account reference patterns before checking for invalid characters.

📌 **Critical Order:** Account reference suffixes (e.g., loan_receivables_prev_balance) must be replaced BEFORE own-row variables (prev_balance). Reversing the order leaves partial tokens that fail validation.

10.4 Management Account Types

Accounts are grouped by type (asset, liability, equity, income, expense). Each type has rules about whether it normally carries a debit or credit balance. This determines how totals roll up.

10.5 Previous Balances API

Each reporting period needs the closing balance from the previous period as its opening balance. A dedicated endpoint fetches previous period balances and injects them into the current period's computed rows.

10.6 Hands-On Tasks

- Open /mnt/project/FIS.xlsb (the reference Excel file) to understand the target GL report format
- Find validateFormula() and trace why account reference suffixes are replaced first
- Write a formula for an account that is the sum of two other accounts' total_balance values

MODULE 11

Fund Transfers & Other Transactions

Duration: 2 Hours • Difficulty: ★★ Intermediate

11.1 Fund Transfers

Fund transfers move cash between branches. They follow a two-step approval flow and are restricted by role.

Role	Permission
finance, regional_manager, deputy_director	Can CREATE fund transfers
rep=3 (Branch Manager)	Can APPROVE/REJECT at branch level
finance shortCode	Final APPROVAL/REJECTION
rep=2 (Area Admin)	Can VIEW only — no approval rights

11.2 Transaction Code Generation

Fund transfers get a unique transaction code in the format FT-YYYYMM-NNNN. The NNNN counter is determined by querying the highest existing counter for that month prefix and incrementing.

11.3 Bad Debt Collections

Accounts that are written off can still have partial payments recorded as bad debt collections. These have their own table (badDebtCollections) and API routes separate from regular cash collections.

11.4 Client Transfers

Clients can be moved between groups (same or different branch). The transfer must:

- Create a transferClients record with status: pending
- Branch Manager approves → update source group noOfClients, update client's groupId, copy cash collection if same date
- All steps execute in a single batch mutation to maintain data consistency

11.5 Hands-On Tasks

- Open src/pages/api/v2/transactions/fund-transfer/ — read save.js, approve.js, list.js in order
- Trace a client transfer approval through src/pages/api/v2/transactions/transfer-client/approve-reject.js
- Verify that the batch mutation pattern is used — find graph.mutation(...mutationList)

🚀 MODULE 12

Deployment & Infrastructure

⏱️ Duration: 2 Hours • Difficulty: ⭐⭐ Intermediate

12.1 Architecture Overview

Service	Technology	Port	Role
lms (x10)	Node.js 18 + Next.js	3000	App containers (scaled to 10 replicas)
nginx	Nginx latest	80 → 3000	Reverse proxy + load balancer
hasura	Hasura v2.37	8080	GraphQL engine
postgres	PostgreSQL	5433	Primary database

12.2 Staging vs Production — Two Different Environments

Aspect	Staging	Production
Process manager	PM2 (Node.js process manager)	Docker Compose (containerized)
How to deploy	Manual — developer triggers it	Script — run ./run.sh on server
Scaling	Single process (PM2)	10 container replicas behind Nginx
Purpose	Test and validate changes	Live user traffic
Branch	migration (or feature branch)	migration (stable, reviewed)

12.3 Staging Deployment (PM2)

Staging uses PM2 — a Node.js process manager that keeps the app running and restarts it automatically on crashes. Deployment is triggered manually by the developer after testing locally.

```
# On the staging server, navigate to the app:
cd ~/apps/lms-v2

# Pull latest code:
git fetch origin migration
git pull origin migration

# Install dependencies (if package.json changed):
npm install

# Build the Next.js app:
npm run build

# Restart with PM2:
pm2 restart lms
```

```
# Check PM2 status:
pm2 status
pm2 logs lms --lines 50
```

📌 **Staging Rule:** Always verify your feature works correctly on staging BEFORE triggering a production deployment. Staging is your last safety net.

12.4 Production Deployment (Docker + ./run.sh)

Production runs via Docker Compose with 10 container replicas behind Nginx for load balancing. Deployment is done by running the run.sh script on the production server. This script handles git pull, build, and container scaling in one command.

```
# SSH into production server:
ssh ubuntu-c-8-sgp1-01-prod

# Navigate to app directory:
cd ~/apps/lms-v2

# Run the deploy script:
./run.sh

# What run.sh does internally:
#   git fetch origin migration && git pull origin migration
#   npm install
#   npm run build
#   docker compose up -d --build --scale lms=10
```

⚠️ **Important:** Never run ./run.sh on staging — it is a production script. On staging, use the manual PM2 steps above.

12.5 Nginx Load Balancing (Production Only)

Nginx routes all traffic to the lms containers using Docker internal DNS (127.0.0.11). It round-robins across the 10 replicas. Timeout is 5 minutes to handle slow Excel export requests.

12.6 PostgreSQL Connection Exhaustion

With 10 replicas each making Hasura queries, PostgreSQL connection limits can be hit. Monitor with:

```
SELECT count(*) FROM pg_stat_activity;
-- max_connections is set in postgresql.conf
```

⚠️ **Warning:** If max_connections is exhausted, Hasura throws "too many clients" and all queries fail. Restart Hasura or reduce replicas temporarily if this happens.

12.7 Environment Variables

- HASURA_ADMIN_SECRET — Must be set; startup aborts if it equals the default value
- HASURA_URL — Internal Docker network URL to Hasura
- NEXT_PUBLIC_API_URL — Exposed to the browser; must point to the public API base
- POSTGRES_* — Database credentials; never commit to git

12.8 Stale Hasura Metadata

After a migration or schema change, Hasura metadata (table relationships, permissions, native queries) can become stale. Fix with:

```
# Via Hasura CLI:
hasura metadata reload
# Or via Hasura console → Metadata → Reload
```

12.9 Hands-On Tasks

- On staging: pull the latest code, build, and restart with PM2 — verify pm2 status shows online
- On production: SSH in, run ./run.sh, then run: docker ps — count the lms containers
- Read nginx.conf — identify the proxy_pass directive and the 5-minute timeout
- Run: docker compose logs --tail=50 lms — look for any runtime errors
- Check Hasura metadata: hasura metadata inconsistency list

Developer Best Practices

Code Patterns

- Always check graph.fields.js before defining new field constants
- Use UNION ALL — never UNION — when combining PostgreSQL subqueries
- Move default objects outside component scope to prevent infinite re-render loops
- Call all hooks before any early return (Rules of Hooks)
- Use useRef flags (hasFetchedRef) to prevent duplicate API calls on mount
- Tailwind dynamic classes are purged in production — add them to safelist in tailwind.config.js

Data Integrity

- Upsert by loanId + dateAdded — never blind INSERT for cash collections
- Batch related mutations — never make sequential graph.mutation() calls for atomic operations
- Use Asia/Manila timezone for all date operations
- Return status 200 with { success: false } for soft failures, never 403 (which triggers auto-logout)

Security

- Prefer req.auth.sub over req.body.currentUserId — the latter bypasses JWT
- Always validate role.rep before any data mutation in API routes
- Never log or expose HASURA_ADMIN_SECRET or database credentials
- Disable Hasura console and dev mode in production

Review Checklist Before Every PR

Check	Why
Role check on every API route?	Prevents unauthorized data access
Batch mutations used where possible?	Atomicity + performance
No hardcoded field strings?	Use graph.fields.js constants
Date operations use Asia/Manila?	Correct business date
New Tailwind classes in safelist?	Classes not purged in production
fetchWrapper used (not raw fetch)?	Auto-logout on 403 works correctly
graph.fields.js checked for duplicates?	No redundant field definitions

Quick Reference Card

Most-Used File Locations

What You Need	File
All GraphQL field definitions	src/lib/graph.fields.js
GraphQL query/mutation utilities	src/lib/graph/graph.util.ts
Hasura client	src/lib/graph/graph.provider.ts
HTTP client (frontend)	src/lib/fetch-wrapper.js
API URL helper	src/lib/constants.js
Redux store	src/redux/store.js
Route protection	src/components/RouteGuard.js
Navigation menu	src/components/Nav.js
DCS data endpoint	src/pages/api/v2/data/get_daily_collection_sheet_v2.js
Cash collection save	src/pages/api/v2/transactions/cash-collection/(saveV2)
Docker config	docker-compose.yml + database/compose-hasura.yaml
Nginx config	nginx.conf

Key PostgreSQL Functions

Function	Purpose
fn_get_daily_collection_sheet_v2	DCS data for a branch + date
fn_get_cash_collection_with_filter_v2	Filtered cash collection list
get_lo_transaction_summary	Loan officer summary totals
get_dashboard_summary_view	Dashboard KPI data

Git & Deployment

```
Branch:             migration
Repo:               git@github.com:evalle012006/lms.git

--- STAGING (PM2) ---
git pull origin migration  &&  npm install  &&  npm run build
pm2 restart lms
pm2 logs lms --lines 50

--- PRODUCTION (Docker) ---
ssh ubuntu-c-8-sgp1-01-prod
cd ~/apps/lms-v2  &&  ./run.sh
docker ps                          # verify 10 replicas running
```